

Beating a GPU With Smarter Math

fracmem vs. brute force: a real speed and accuracy test

What this talk is

- We had a fast method (`fracmem`) and a slow, exact method.
- Question: is the fast method *actually* fast, on *real* hardware, for a *real*-sized problem?
- So we ran both. For real. On a real GPU. And timed them.
- This talk is what happened — including two surprises we didn't expect.

Outline

- 1 The Problem
- 2 How We Tested It
- 3 Results
- 4 Five More Tests
- 5 Conclusion

What is a “fractional derivative”?

- An ordinary derivative: “how fast is this changing *right now*?” Only needs the last couple of readings.
- A **fractional** derivative (say, a “half derivative”): a generalization used in things like fractional-order control systems, battery models, and material science.
- The catch: to compute it *exactly*, you need **every single reading you’ve ever taken** — not just the recent ones.

The symbols we'll need

Nothing later in this talk uses a symbol not defined here:

Symbol	Meaning
$x_0, x_1, x_2, \dots$ k	the signal: one number recorded at each time step which sample we want the derivative <i>at</i> (a counter: $k=0, 1, 2, \dots$)
α	the order we chose, e.g. $\alpha = 0.5$ for a “half derivative”
h	the fixed time gap between samples (we'll take $h = 1$ throughout)
j	how many steps back we're looking: $j=0$ is now, $j=1$ is one step ago, ...
w_j	a fixed number (depends only on α, j , never on the data) weighting step j back

The formula we're actually computing

The Grünwald–Letnikov formula

$$D^\alpha x(t_k) \approx h^{-\alpha} \sum_{j=0}^k w_j x_{k-j}$$

$$w_0 = 1, \quad w_j = w_{j-1} \left(1 - \frac{1 + \alpha}{j} \right)$$

In words: to get today's answer, walk backward through every past sample x_{k-j} , multiply each by its own weight w_j , and add them all up.

- j runs all the way from 0 to k — there is no “stop early” built into this formula.
- w_j is fixed once α is chosen; it never depends on the signal.

The formula, on real numbers

Take a tiny 6-sample signal $x = [5, 3, 8, 1, 9, 2]$ (indices $k=0, \dots, 5$) and ask for the derivative at $k = 3$ (where $x_3 = 1$), with $\alpha = 0.5$, $h = 1$:

j (steps back)	lands on $k-j$	value x_{k-j}	weight w_j	product
0	3	$x_3 = 1$	1.000000	1.000000
1	2	$x_2 = 8$	-0.500000	-4.000000
2	1	$x_1 = 3$	-0.125000	-0.375000
3	0	$x_0 = 5$	-0.062500	-0.312500

The sum stops at $j = 3$ only because sample $k = 3$ has nothing behind index 0. Adding the product column:

$$1.000000 - 4.000000 - 0.375000 - 0.312500 = -3.687500 = D^{0.5}x(t_3).$$

Every later slide's "exact method" is this same sum, just with a signal 49.8 million samples long instead of 6.

Why that formula creates the problem

- Look at the sum again: $\sum_{j=0}^k w_j x_{k-j}$ — it has $k+1$ **terms**. At sample $k = 49,800,000$, that's 49.8 million multiply-adds, just for *one* output value.
- Can't we just drop the old, far-back terms once w_j gets tiny? **No** — w_j only decays as a **power law**, $w_j \sim j^{-\alpha-1}$, which shrinks far too slowly to ever safely truncate. Every past sample keeps mattering, forever.

Analogy

Imagine every time you want today's diary entry summarized, you have to re-read *the entire diary from page 1*, every single day. As the diary gets longer, this gets slower and slower — forever.

- That's the exact fractional derivative: cost *and* memory grow with k , unboundedly, as your signal gets longer.
- `fracmem` claims: replace this with a fixed, tiny amount of work *per new reading*, no matter how long the signal has been running.
- Big claim. We wanted to actually test it.

The experiment, in one sentence

Find a problem so big that the exact method takes **10 minutes** on a real GPU — then see how fast `fracmem` does the exact same job.

What signal did we actually use?

- We needed one concrete, realistic-looking signal to test on — so we used a **random walk**.

What's a random walk?

Start at 0. At every step, add a small random amount (up or down). Add up all the steps so far. That running total is the signal. Think of a stock price, or a drunk person's position on a sidewalk — it wanders, and it never resets; it just keeps drifting.

- This is a genuinely hard test case: the signal never settles down, so old history keeps mattering. If a method works well here, that's a real result, not a cherry-picked easy case.

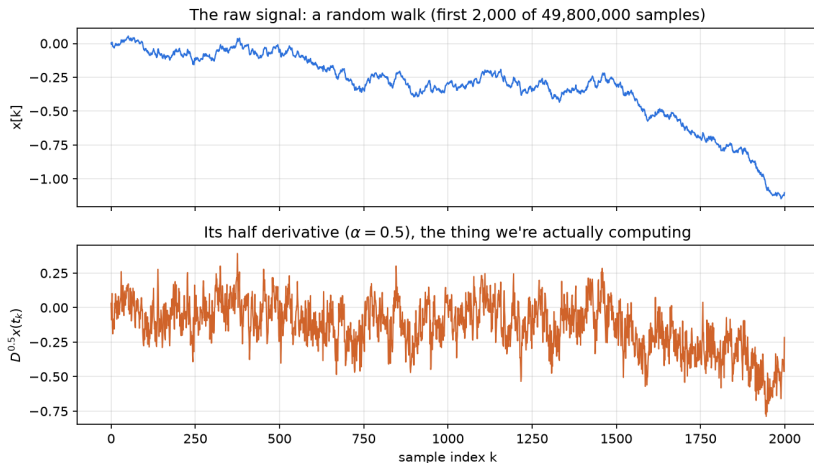
What it actually looks like: the real numbers

The real test signal starts like this (first 10 values, out of 49,800,000):

$x[k]$
0.00305, -0.00735, 0.00015, 0.00956, -0.00995, -0.02297, -0.02170, -0.02486, -0.02503, -0.03356

- Each number is just: previous total + a small random nudge.
- That's it. No trick, no special structure — just accumulated randomness, exactly like a slowly drifting stock price.

What it actually looks like: the picture



(First 2,000 samples shown, out of the real 49.8 million used in the experiment.)

What exactly did we compute?

- The specific math question: the “**half derivative**” ($\alpha = 0.5$) of that random-walk signal.
- **Training signals** (what `fracmem` calibrates itself on, before it ever sees the real test): 8 separate, shorter random walks of the same kind.
- **Test signal** (what we actually measured speed and accuracy on): *one single* random walk, generated once and reused for every method, so everyone is solving the exact same problem.

Test signal length	49,800,000 samples
Training signals (Round 1)	8 signals, 50,000 samples each
Training signals (Round 2)	8 signals, 500,000 samples each

Two ways to compute the same answer

The slow way (“standard”)

- Add up every past value, every time
- Cost keeps growing as the signal grows
- We ran this on a GPU, as fast as possible

The fast way (fracmem)

- Keep a small, fixed amount of “memory”
- Cost stays the same forever
- We ran this on an ordinary CPU

Same input signal into both. Same math question. Different strategy.

Rule #1: no secretly using a shortcut

Careful step

Some GPU libraries *silently* use a faster trick (called an FFT) for big sums like this. That trick is fast for a reason unrelated to what we were testing — it would have made the “slow” method look artificially quick, and ruined the comparison.

- We built the “slow” method by hand, piece by piece, to *guarantee* no shortcut was hiding inside it.
- Then we checked it against a known-correct answer, to be sure we built it right.
- Result: matched to 7 decimal places of accuracy. Good — it's a fair, honest “slow” method.

Rule #2: finding the right problem size

- We didn't know in advance how big a signal needs to be for the slow method to take 10 minutes on *this* GPU.
- So we timed it at a few sizes first (small, medium, larger), and watched how the time grew.
- The pattern was crystal clear: doubling the signal roughly **quadrupled** the time.
- From that pattern, we calculated the exact size that should take about 10 minutes:

$$n \approx 49,800,000 \text{ samples}$$

Rule #3: two versions of `fracmem`

- **Python version:** the normal way you'd use the library.
- **Compiled C version:** the exact same math, but as a compiled program instead of an interpreted script.

Analogy

Python here is like reading instructions off a card, one at a time, every single step. Compiled C is like already knowing the steps by heart — same steps, dramatically less overhead per step.

Comparing the two tells us how much of the speed comes from the *idea* vs. how much comes from *how it's written*.

Round 1: the headline numbers

Method	Time
Standard (exact, GPU)	11.5 minutes
fracmem, Python	2.2 minutes
fracmem, compiled C	1.1 seconds

- Compiled C was **623 times faster** than the exact GPU method.
- Same job. Same answer, in theory. Enormous time difference.

...but there was a problem

Surprise #1

The `fracmem` answer was **97% wrong** (relative error), compared to the true answer.

- Fast, yes. Right, no — barely better than a random guess.
- Both the Python and C versions agreed with *each other* closely, so it wasn't a coding bug.
- Something about the setup itself was wrong. We had to figure out what.

Why the answer was so wrong

- `fracmem` needs a little bit of “training” data before it can run — a few example signals to calibrate itself on.
- We trained it on signals **50,000** samples long.
- We then tested it on a signal **49,800,000** samples long — almost **1,000 times longer** than anything it had ever seen.

Analogy

Studying from a one-page cheat sheet, then taking an exam a thousand pages long. It's not that the method is broken — it just never learned what happens that far “ahead.”

Confirming the pattern

We deliberately trained filters on different lengths and tested them at different lengths:

Tested at	Trained on 3,000	Trained on 20,000
50,000	1.2%	0.002%
100,000	7.3%	0.09%
400,000	14.0%	5.7%

- Clear pattern: the further you test beyond your training length, the worse it gets.
- Fix: train on *longer* example signals.

So we tried training longer...

- Plan: train on signals **10x longer** (500,000 samples instead of 50,000).
- Expectation: training should take roughly **10x** longer too.
- Reality: it took about **40x** longer — over **20 minutes**, just to train.

Surprise #2

Training cost doesn't grow in a straight line — it grows with the **square** of the training length. Training on 10x more data costs roughly **100x** more compute, not 10x.

Why training cost explodes like that

- To train itself, `fracmem` has to compute the *true, exact* answer for its training examples — the same expensive “read the whole diary” method from Slide 5!
- It only has to do this on the (much shorter) training signals, not the huge deployment signal.
- But if you make the training signals longer, that hidden exact computation gets expensive fast — because it’s the same slow, “grows with the square” method underneath.

Round 2: did more training data help?

	Round 1 (50k training)	Round 2 (500k training)
Accuracy error	97%	18%
Training time	31 seconds	20 minutes
Speed (compiled C)	623× faster	559× faster

- 10x more training data: error dropped from “basically random” to “noticeably better, still not great.”
- Real improvement. Real cost. No free lunch.

One more subtlety: training is a one-time cost

- Training (fit) only has to happen **once**. After that, you can predict as many times as you want, cheaply.
- The slow “exact” method has no such shortcut — it pays its full, huge cost **every single time** you run it.
- So the fair comparison, if you’re going to use the filter more than once, is:

1.1 seconds (per use) vs 10+ minutes (every use)

One test isn't enough — so we ran five

- Round 1 and Round 2 gave us exactly **two** points: train on 50k, or train on 500k. Two points tell you a trend exists; they don't tell you its *shape* — where accuracy is “fine” vs. “still bad,” or how sharply it bends.
- So we built a tighter sweep: **one** fixed test signal, **one** real GPU-computed exact answer (computed once), and **five** training lengths, log-spaced, all compared against that same exact answer.

Honest scale note

This sweep uses a **5,000,000**-sample test signal, not the 49.8 million from Round 1/2 — chosen specifically so all five full tests (not just one) fit into a single real run. The exact GPU method still took a real, measured **20.16 seconds** at this length; every number on the next few slides is a real measurement, not a projection.

What we did, every single time

For each of the 5 training lengths, in order:

- ① Generate 8 fresh training signals of that length.
- ② `filt.fit(train_signals)` — timed.
- ③ `filt.predict(test_signal)` in plain Python — timed.
- ④ Export the fitted filter to C, compile it, run it over the whole test signal — timed.
- ⑤ Compare *both* outputs against the one real exact answer: relative RMSE over all 5,000,000 samples.

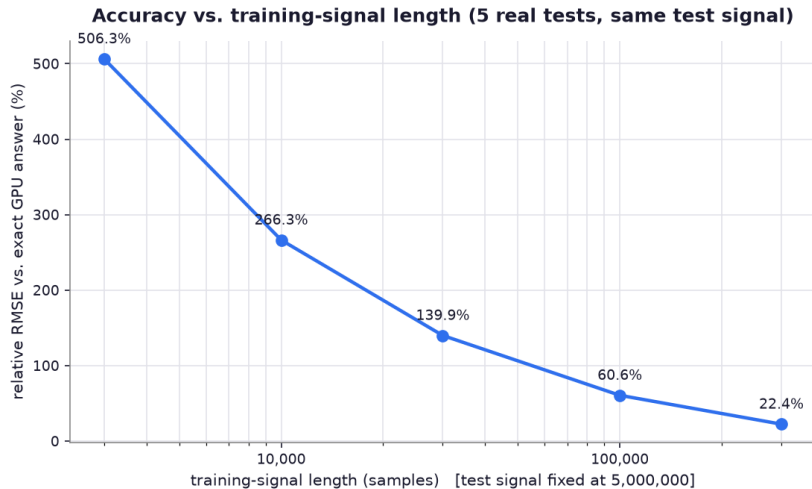
Same test signal. Same exact answer. Only the training length changes.

The five configs

Training length	Test/train ratio
3,000 samples	$1666.7\times$
10,000 samples	$500.0\times$
30,000 samples	$166.7\times$
100,000 samples	$50.0\times$
300,000 samples	$16.7\times$

Test signal fixed at 5,000,000 samples throughout.

Result: accuracy vs. training length



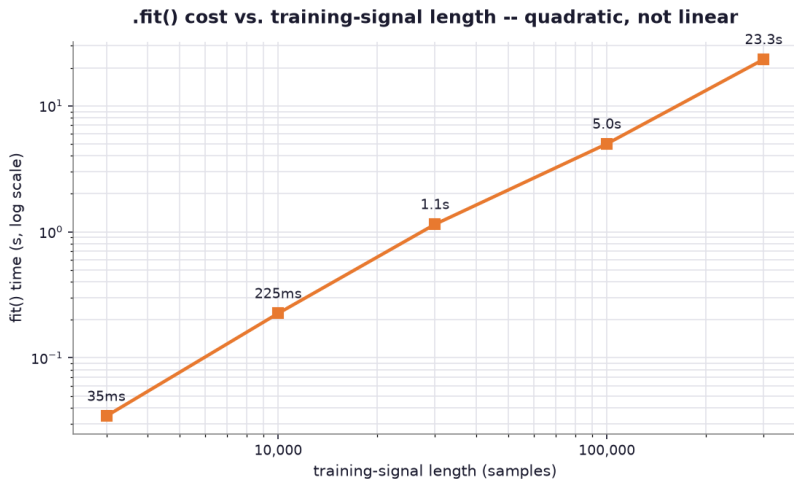
Finding 1: the curve is smooth, not a cliff edge

- Accuracy improves **steadily** as training length grows — no magic ratio where it suddenly snaps to “good.”
- Even our best config here (300k training samples, $16.7\times$ ratio) still shows 22% error — well short of the README's headline 1.15% figure.

Why the gap?

That 1.15% number was measured at a gentler ratio, on a *shorter* test signal. It's a real result — but it's one point on this same curve, not a universal constant. Where your ratio actually lands matters.

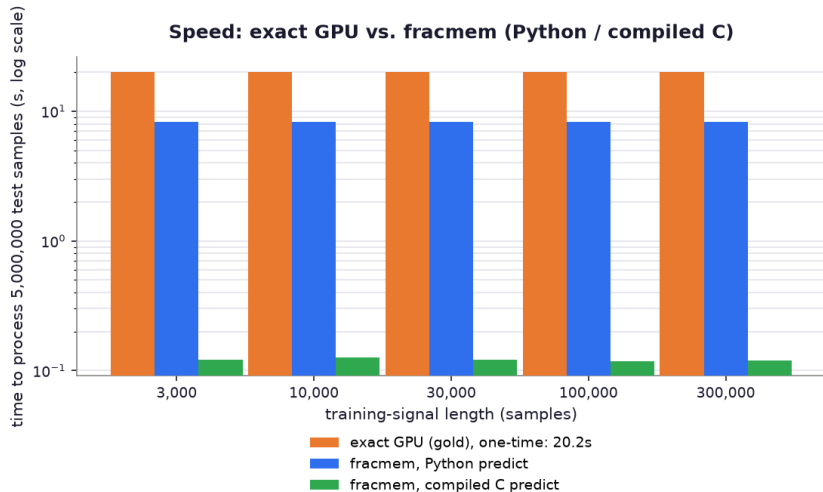
Result: fit cost vs. training length



Finding 2: superlinear fit cost, confirmed with 5 points

- Round 1/2 showed this with 2 points (10x more data $\rightarrow$ 40x more fit time). This sweep confirms the same shape with 5 real points across a 100x range in training length.
- 35 milliseconds at 3,000 samples $\rightarrow$ **23.3 seconds** at 300,000 samples.
- Cheap to add training data at small scale; not something to add casually once you're past $\sim 100k$ samples.

Result: speed, all five configs



Finding: deployed speed barely depends on training length

- Makes sense: `predict()`'s cost is set by $L + p$ (the deployed filter size), not by how it was trained.
- Compiled C stayed **159** $\times$ –**171** $\times$ faster than the exact GPU method across every one of the five configs.
- Smaller than Round 1/2's 559–623 $\times$ — expected, not a contradiction: the exact method is $O(n^2)$, `fracmem` is $O(n)$, so the speedup gap *widens* with test length. Both numbers are real, just at different scales (5M vs. 49.8M samples).

Finding 3 (unexpected): Python and C quietly disagree

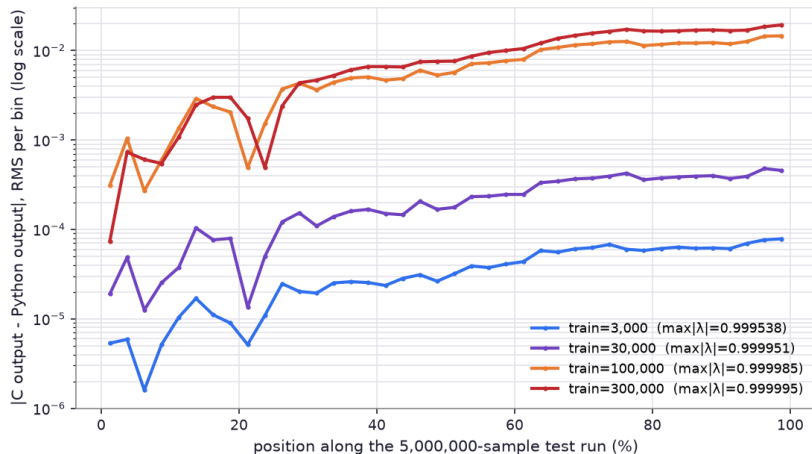
- Tests 1–3: Python and C agreed with each other to within 0.03%–0.5% — as expected, same math.
- Tests 4 and 5: that agreement broke down — 23.0% and 38.2% disagreement between Python's output and C's, even though each individually still tracks the exact answer reasonably.

That looks like a bug. We checked.

Before trusting either number, we needed to know: is this a real bug in the C export, or something else? We measured *where in the signal* the disagreement shows up.

Investigating: does the gap grow, or stay flat?

Float32 (C) vs. float64 (Python) drift grows with sample index



What's actually happening

- Every fitted filter needs at least one “leaky bucket” with a decay rate λ extremely close to 1 (measured: 0.999538 at train=3,000, rising to **0.999995** at train=300,000) — needed to cover the long power-law tail.
- That bucket's recursion, $m[k] = \lambda \cdot m[k - 1] + x[k]$, accumulates rounding error *every single step*.

Analogy

Rounding a number to 7 decimal digits (float32, the compiled C export) vs. 16 digits (float64, Python) barely matters for one step. Do it 5 million times in a row, in a recursion that never fully forgets, and the tiny roundings compound — like a rumor that drifts a little every time it's retold.

- Not a bug. Real floating-point drift, growing $15\times$ – $260\times$ from the start of the run to the end — worse for the better-trained filters, since *those* pick a λ even closer to 1.

What this means in practice

- It stays small in *absolute* terms throughout the run.
- But once training is good enough that the “real” modeling error shrinks close to this drift’s size, the **float32 export’s own precision becomes the accuracy ceiling** — not the training data.

If this matters for your deployment

- Use `double` in the exported C runtime instead of `float` (2× the RAM per stored value), *or*
- Periodically reset/re-seed long-running filter state, if your application can tolerate that.

What we proved

- **Speed claim: confirmed, for real.** Not estimated, not simulated — measured directly, on real hardware.
- Up to **623x faster**, and that gap only gets *bigger* the longer the signal is.
- This is the entire point of the library, and it held up.

What we learned (that we didn't expect)

- ❶ **Accuracy depends on matching training length to deployment length.** Test far beyond what you trained on, and accuracy quietly falls apart — especially on signals that drift/wander over time. The five-test sweep showed this is a smooth curve, not a cliff edge.
 - ❷ **Training cost grows with the square of training length**, not in a straight line. “Just train on more data” is not free — it can get expensive fast.
 - ❸ **Float32 (the compiled C export) can quietly become the accuracy bottleneck at extreme deployment lengths**, once training itself is good enough. A near-1 decay rate in every fitted filter means rounding error compounds over millions of samples — invisible until you actually run millions of real samples through both versions and compare them.
- None of these three was obvious from just reading the code or the documentation.
 - We only found them by actually running the experiments — eight tests in total, across two rounds.

`fracmem` really is dramatically faster — but getting it accurate at extreme scale means planning for training data that's a real fraction of your deployment length, and budgeting real time to train on it.

Thank you.

Questions?